TPIT AI ACADEMY 2026 · BASIC CURSOR AI FOR SOFTWARE DEVELOPER

# Session 1: Introduction to Agentic AI Software Development

Instructor: Thanapat Bhunnachet (Tob)



# What you will be able to say after today

01 **Name the keywords** of agentic AI development

02 Know which session teaches each one

03 Turn a 3-line requirement into **a spec, a plan, and a running file**

# This is where we are going

- Look at the finished demo: **the equipment loan list**
- We build it live today, from a plain requirement
- Later projects are different. **The ideas carry over:** say what you want, plan before code, check every step

# Quick check: is Cursor working?

01 Open Cursor and start a new chat

02 Type **Say hello in one sentence** and send it

03 You get an answer: you are ready

— No answer? Watch the screen and pair with a neighbor

— We fix installs at the end, after the content

# Fundamentals





# A short history: 2 minutes

1990s on

## Traditional autocomplete

The editor finishes a name it already knows. Rules and symbol tables, no model

2021 on

## Small-model completion

A small, fast model predicts the next lines. Example: GitHub Copilot

2023 on

## Context-aware chat

You ask in plain language. The model reads the files you show it and answers or edits

Late 2024 on

## Agents

The model plans, edits files, runs commands, reads the result, and repeats





# Recap: LLM (Large Language Model)

FROM AI FOUNDATION

A model trained on a vast amount of text, that predicts the next word. A very good autocomplete

โมเดลภาษาขนาดใหญ่ — โมเดลที่เรียนรู้จากข้อความจำนวนมาก แล้วทำนายคำถัดไป

ONE MORE THING FOR ENGINEERS

- **Pure function:** text in, next token out
- No memory between calls. It never runs your code
- **The harness** is the program around the model. It resends the context, calls tools, and runs commands. Cursor is a harness. GitHub Copilot is another one

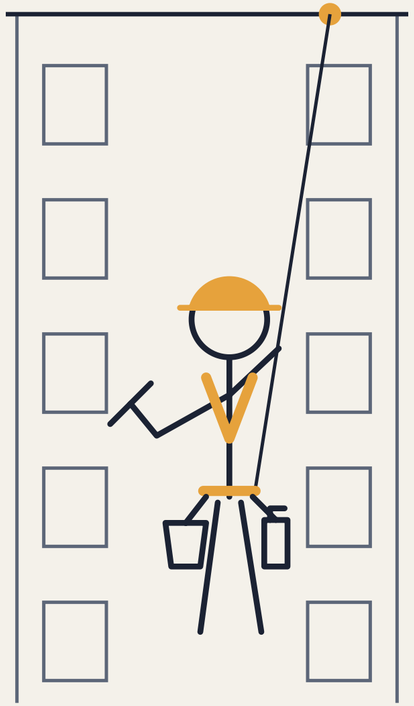
ฮาร์เนส — โปรแกรมที่ครอบโมเดล ส่งบริบท เรียกเครื่องมือ และรันคำสั่งให้





# Keyword: harness

| ON THE BUILDING | IN AGENTIC AI |
| --- | --- |
| The brain | **The model:** it thinks and decides, then sends signals. It cannot touch anything |
| The body, plus the harness it wears | **The harness** (today: Cursor): it follows the signals, reads and writes files, and runs commands |
| Gear clipped on with karabiners | Tools, rules, skills, MCP servers |
| The safety line to the roof | **Your approvals.** Auto-run off |



# Recap: token

FROM AI FOUNDATION

A small piece of text, close to a word or part of a word

RULE OF THUMB

- 1 token is about 4 English characters (4 bytes)
- 1,000 tokens is about 750 English words (4 KB)

โทเคน — หน่วยย่อยของข้อความที่โมเดลใช้อ่านและเขียน

ONE MORE THING FOR ENGINEERS

- Code and Thai **cost more tokens per character** than English
- Thai is 3 bytes per character in UTF-8. The 4-byte rule does not work for Thai
- More tokens: more cost, slower, less room
- Long chats with big pasted files get slow, expensive, and fill the context window (next)





# Recap: context window

FROM AI FOUNDATION

The amount of text the model can hold at once

COUNTED IN TOKENS

For example, 256K tokens is about 1 MB of English text (slide 9, "Recap: token")

ขนาดหน้าต่างบริบท — จำนวนโทเคนที่โมเดลอ่านได้ในครั้งเดียว

ONE MORE THING FOR ENGINEERS

- One window holds: system prompt, tool definitions, rules, skills, MCP tools, and the conversation
- **Every message resends the whole chat**: your prompts, @-files, and every tool result (slide 7, "Recap: LLM")
- When it fills, the harness summarizes or drops older parts
- So: **start a new chat per task**, and **@-mention only what matters**

# Recap: hallucination

FROM AI FOUNDATION

A confident answer that is wrong

อาการหลอน — โมเดลตอบสิ่งที่ฟังดูมั่นใจแต่ไม่จริง

READJUST

- **The model is not confident.** It writes the most likely next token
- No "I am not sure" signal, and no tone or body language: a smooth answer reads as certain
- It is trained to write fluent prose, and the chat hides the token probabilities

# Recap: hallucination

FROM AI FOUNDATION

A confident answer that is wrong

อาการหลอน — โมเดลตอบสิ่งที่ฟังดูมั่นใจแต่ไม่จริง

ONE MORE THING FOR ENGINEERS

- The common shape: **gap filling**
- The model has no file, doc, or data value, so it guesses
- The guess looks like a real API, flag, or path
- Often the right file was available, but nobody showed it to the model

## The fix is not "ask again".

- Show the model the real file or docs
- Run the code: the compiler or test tells the truth





# Keyword: non-deterministic

THE KEYWORD

The same prompt can give a different answer each time. This is **normal model behavior, not a bug**

ผลลัพธ์ไม่คงที่ — ถามคำถามเดิมสองครั้ง อาจได้คำตอบต่างกัน

ONE MORE THING FOR ENGINEERS

- The model **samples** the next token. It does not pick the single best one
- Same dice roll as slide 11, "Recap: hallucination": a likely token, not the one right answer
- Do not expect the same code twice
- **Pin behavior with a spec and a test.** Compare plans, not first impressions



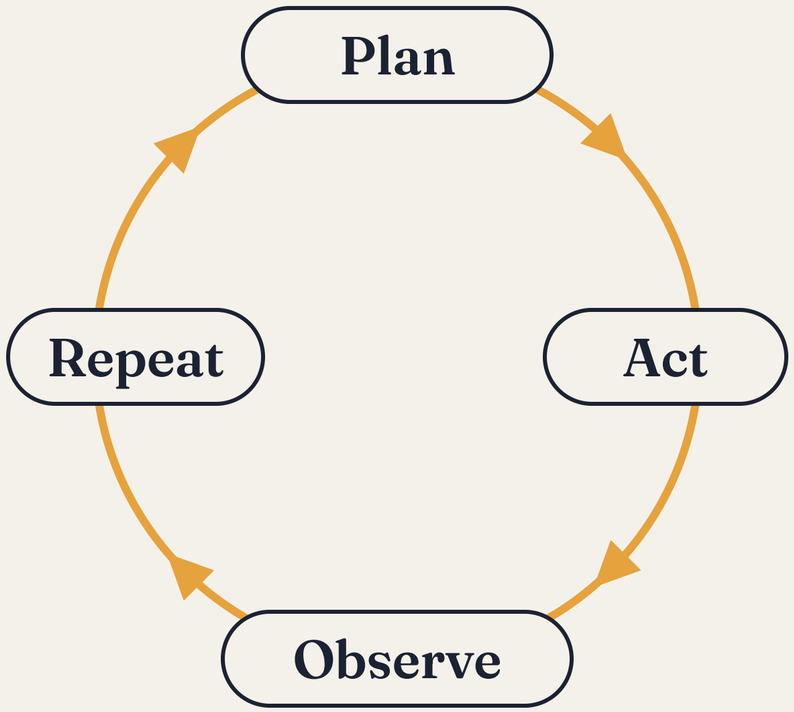


# Keyword: tool use

- **The model cannot run anything.** It only writes text (slide 7, "Recap: LLM")
- **In:** the harness puts the tool list into the context (name, purpose, inputs)
- **Out:** the model is trained to write a tool call: the tool name plus its inputs
- **Back in:** the harness runs the call and pastes the result into the context
- The brain sends the signal. The body runs it (slide 8, "Keyword: harness")

การใช้เครื่องมือ — โมเดลเขียนคำขอ แล้วฮาร์เนสเป็นผู้รันให้

# Keyword: agent loop




- **Plan, act, observe, repeat**
- Your own loop: read, write code, run the test, read the result, fix
- The agent runs the same loop, with tools (slide 14, "Keyword: tool use") in place of your hands

วงจรเอเจนต์ — วางแผน ลงมือทำ ดูผล แล้ววนซ้ำ

# Keyword: agent loop

- How does it know it is right? **It runs the tests and reads the output**
- No test and no error output: nothing to observe, so it guesses (slide 12, "Recap: hallucination")
- Each turn adds a tool result, so a long run fills the window (slide 10, "Recap: context window")
- It ends with a final answer, or when you stop it

# Keyword: autonomy levels

1990s on

## Traditional autocomplete

### No AI

No autonomy. Same input, same output (slide 13, "Keyword: non-deterministic")

2021 on

## Small-model completion

### Completion

Completes code as you type. In Cursor: Tab

2023 on

## Context-aware chat

### Ask

Answers questions, edits no file

Late 2024 on

## Agents

### Agent

Plans and runs many steps on its own

**An agent is a model plus a harness that runs the loop** (slides 14 and 15, "Tool use", "Agent loop")





# The cost of autonomy

- More autonomy means **more steps you did not read**
- AI Foundation, module 3: AI suggests, **you decide, and you are responsible**
- **Pick the lowest level that does the job**
- **Read and verify** before you accept: the agent checks its tests, you check the agent

# Cursor tour

# What you will open

- **Two windows:** Agents (home) and IDE (the editor). We use the IDE today
- **Tab, Ask, Agent**
- @-context, project rules, model picker, Context Usage panel
- Cursor is our first harness, not the only one. GitHub Copilot has the same Ask and Agent modes
- Follow along on your own machine. **Did the setup check? This is a repeat**

# Try it yourself now

01 Click **IDE** at the top right. The editor opens

02 No `ai-academy-demo` folder yet? **Open project** > **Documents** > **New Folder**. Click **Open**, then **Yes, I trust the authors**

03 **Tab**: open or create `hello.html`, type `<p>Made with`, press Tab to accept

04 **Ask**: pick Ask in the mode list, ask one question about hello.html

05 **Agent**: pick Agent in the mode list. Do not run it yet





# Live demo





# The equipment loan list

- A small internal tool: who borrowed which item, and when it is due
- Same Thai requirement as the tour, now in **Agent**: this time files appear
- Watch for 4 moments: **spec first, plan first, one step then check, error pasted back**
- Why: the model's code is **very likely correct, not guaranteed**

# Milestone: spec, plan, generate, run

01 Ask for a spec before any code (`spec.md`)

02 Ask for a step-by-step plan (`plan.md`)

03 Generate step 1 only

04 Open the file in your browser. Check step 1 against the plan

05 Then generate the rest

# Milestone: break it, then fix it

- We rename one function on purpose
- **Paste the console error back into the chat, nothing else**
- Watch the model find the cause and fix it
- Then `@spec.md`: record the return date. **Show the file, so it does not guess**





# Mini hands-on





# Your turn: pick one requirement

01 Open `hands-on.md` at `https://tnptop.com/ai-academy/2026/s1/`

02 Pick one: **meeting room booking list, visitor log, or team to-do list**

03 Open `ai-academy-demo`. Start a **new chat** in Agent mode

04 Paste the requirement **and** prompt (a) in one message. Then prompt (b)

05 Generate, then open the file in your browser

# Reflection questions

01 Where did the plan differ from what you expected?

02 What did the agent decide on its own that your requirement did not say? For example the date format, or what happens on delete

03 If something failed when you ran it, what did you do next?

# Roadmap and peek

# Concept map: where each keyword goes next

| | | |
|---|---|---|
| 1 | How the model works | Session 3 |
| 2 | Chat vs agent | Sessions 2 and 3 |
| 3 | Context engineering | Session 4 |
| 4 | Spec-driven workflow | Sessions 2 and 5 |
| 5 | Verification loop | Session 4 |
| 6 | Extending the agent | Session 4 |
| 7 | Responsible use | Common Course |

# Peek: MCP (Model Context Protocol)

Without MCP, you copy a ticket from the ticket system and paste it into the chat.

With MCP, **the agent reads the ticket itself**.

- MCP is the standard that makes this work: a server exposes tools and data, the agent calls them. An MCP server is gear that clips onto the harness
- Company rule: **only MCP servers IT has approved**, even for a prototype. Ask your UM for the list
- Full depth in Session 4

# Peek: skills and commands

- **A skill is a saved procedure in a Markdown file** (`SKILL.md`). Example: "review this file: run the linter, list each problem, propose a fix"
- The agent starts a skill by itself when the task matches. **Rules are always on** (from the tour)
- Skills are an open standard. They clip onto the harness
- A command is a saved prompt. You start one with / in the chat. Most harnesses have skills and commands
- Full depth in Session 4

# Peek: responsible use

- AI Foundation rules still apply: classify data before you paste, no secrets or personal data, verify before you use. **You own the decision**
- New with an agent: **it acts, not only answers.** Read every command before you approve it. **Keep auto-run off**
- CIO-certified at every company: Cursor (all its models), Gemini, GitHub Copilot. Codex and Claude Code are not on the list. Your company IT may allow them. Ask your UM
- Use your company Cursor account only. No secrets in prompts, rules files, or code

# Close

# Homework and Q&A

- Finish your Cursor install and run the quick check **before 2026-11-05**
- Homework and the hands-on sheet stay at `https://tnptop.com/ai-academy/2026/s1/`
- Write **one capstone idea in 5 lines**: a twist on a later session's project, or a fresh idea from your own work
- Questions now, or ask in the course Google Chat group: `https://chat.google.com/app/chat/AAQAEJ69C4A`

